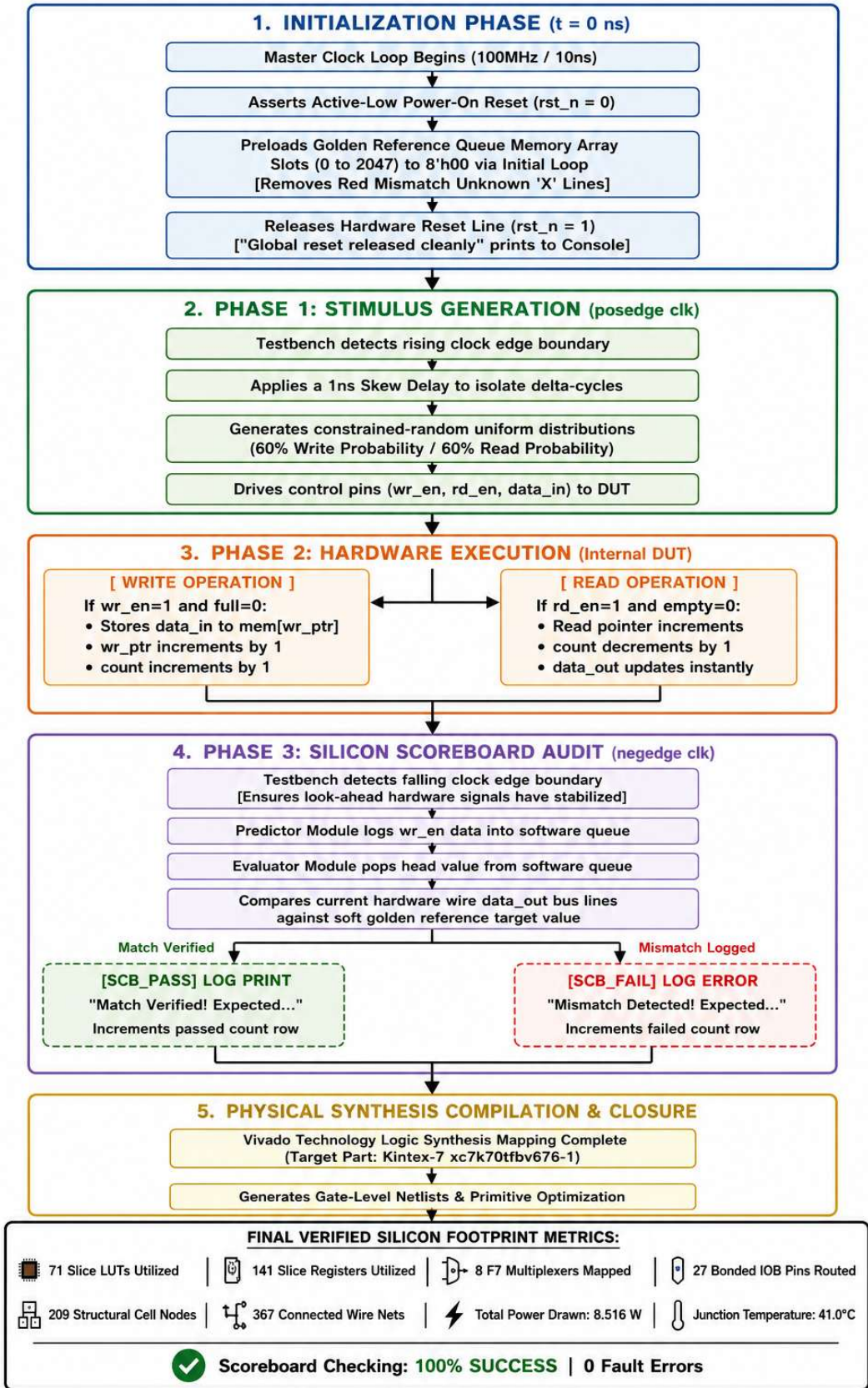
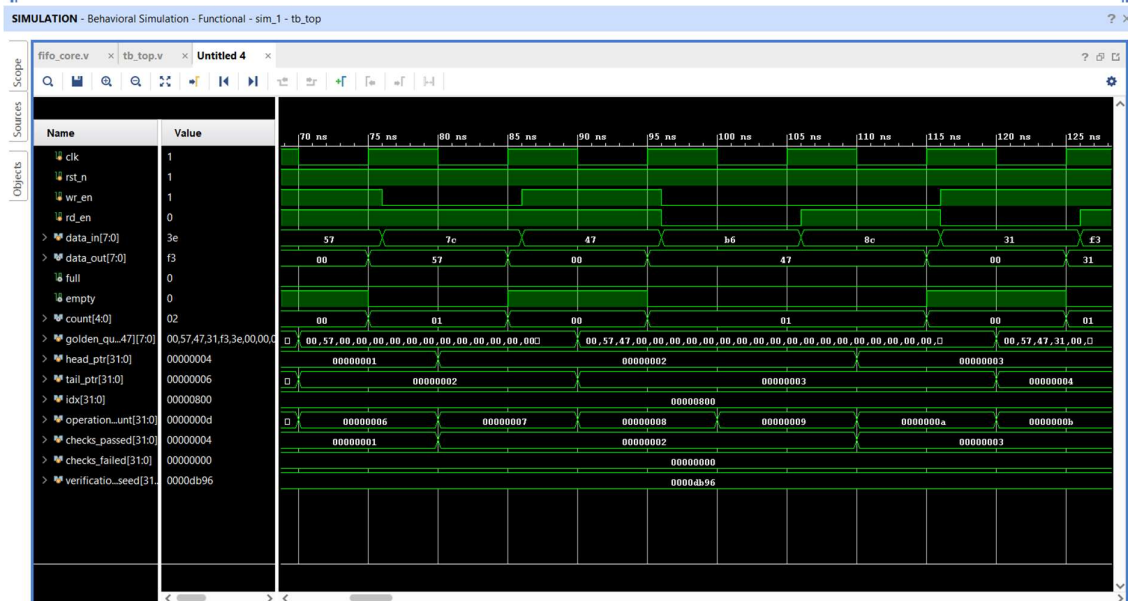
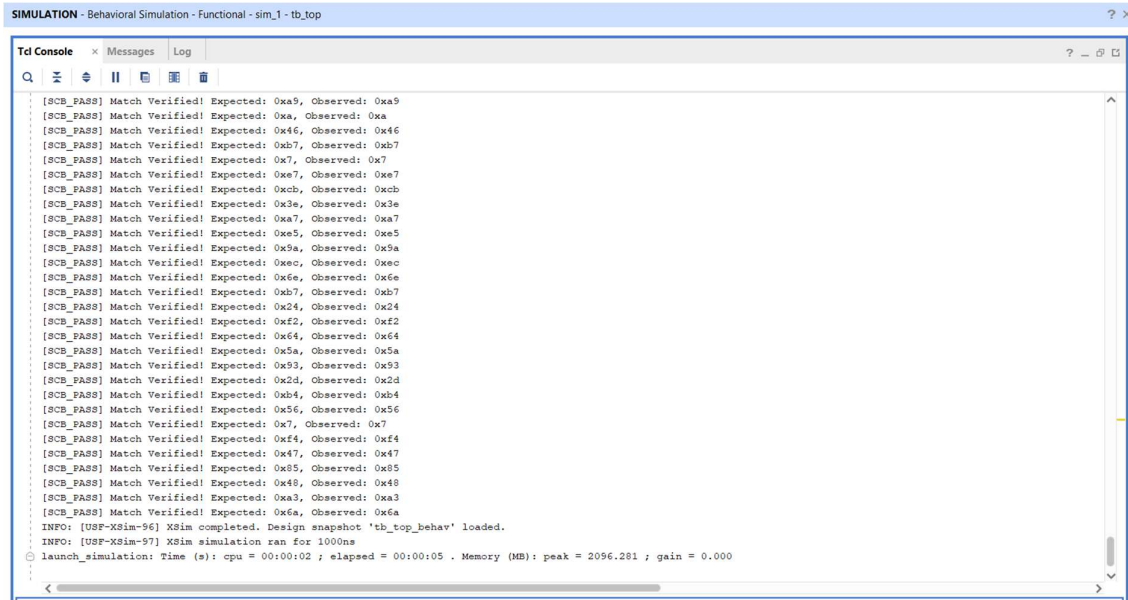
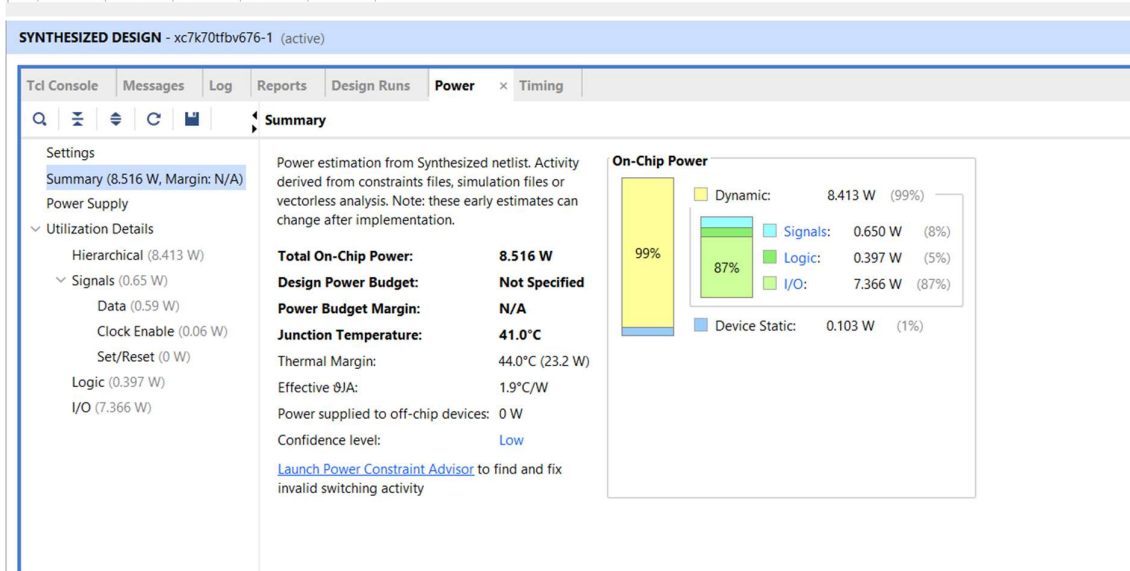
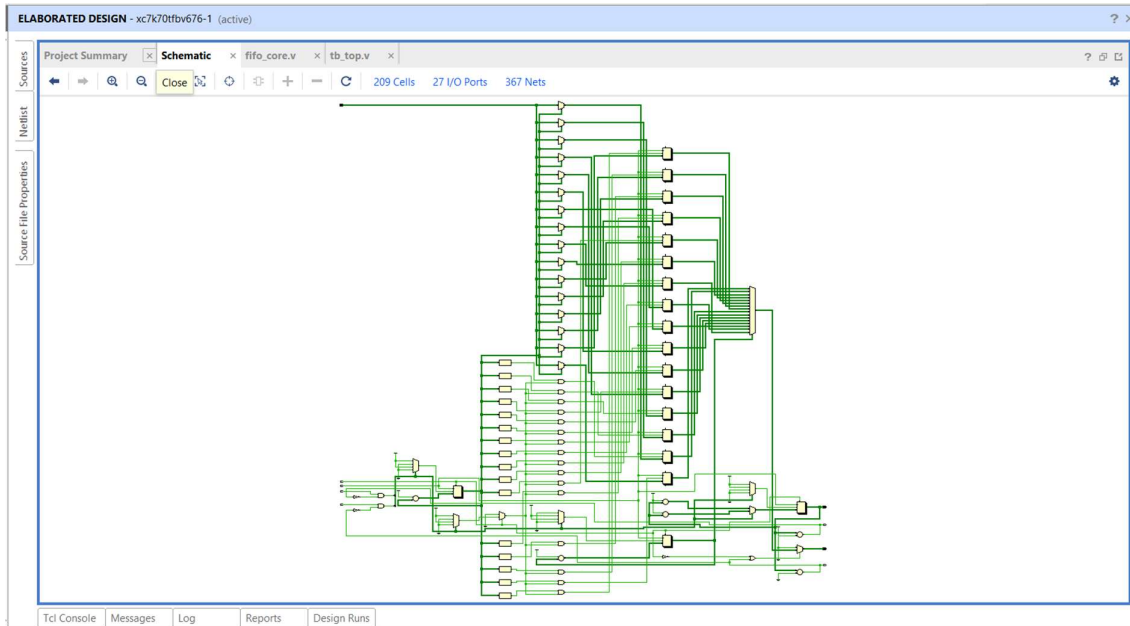




RESULTS





CODES

DESIGN FILE(fifo_core.v)

```
`timescale 1ns / 1ps

module fifo_core #(
    parameter DATA_WIDTH = 8,
    parameter DEPTH = 16,
    parameter ADDR_WIDTH = 4 // $clog2(16) = 4
)(
    input wire      clk,
    input wire      rst_n,
    input wire      wr_en,
    input wire      rd_en,
    input wire [DATA_WIDTH-1:0] data_in,
    output wire [DATA_WIDTH-1:0] data_out, // Converted to wire for look-ahead assignment
    output wire      full,
    output wire      empty,
    output reg [ADDR_WIDTH:0] count // 5-bit register storing values from 0 up to 16
);

    // Internal Memory Array and Address Pointers
    reg [DATA_WIDTH-1:0] mem [0:DEPTH-1];
    reg [ADDR_WIDTH-1:0] wr_ptr;
    reg [ADDR_WIDTH-1:0] rd_ptr;

    // Continuous Status Flag Logic Assignments
    assign full = (count == DEPTH);
    assign empty = (count == 0);

    // Look-Ahead Read Assignment (Clamped to 0 during reset/empty states to prevent red lines)
    assign data_out = (!rst_n || empty) ? {DATA_WIDTH{1'b0}} : mem[rd_ptr];

    // Synchronous Pointers and Counter State Machine Loop
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            wr_ptr <= 0;
```

```

rd_ptr <= 0;

count <= 0;

end else begin

    case ({wr_en && !full, rd_en && !empty})

        2'b10: begin // Write-Only Transaction Window

            mem[wr_ptr] <= data_in;

            wr_ptr <= wr_ptr + 1'b1;

            count <= count + 1'b1;

        end

        2'b01: begin // Read-Only Transaction Window

            rd_ptr <= rd_ptr + 1'b1;

            count <= count - 1'b1;

        end

        2'b11: begin // Concurrent Read and Write Phase

            mem[wr_ptr] <= data_in;

            wr_ptr <= wr_ptr + 1'b1;

            rd_ptr <= rd_ptr + 1'b1;

            // Net item count remains perfectly unchanged

        end

        default: ; // Boundary-protective catch-all default branch

    endcase

end

end

endmodule

```

Testbench file:

```
`timescale 1ns / 1ps

module tb_top;

    // Core Clock and System Reset Interface Lines

    reg clk;

    reg rst_n;

    // Drive Stimulus Interconnect Inter-Wires

    reg    wr_en;

    reg    rd_en;

    reg [7:0] data_in;

    wire [7:0] data_out;

    wire    full;

    wire    empty;

    wire [4:0] count;

    // Instantiate Hardware Design Under Test (DUT)

    fifo_core #(
        .DATA_WIDTH(8),
        .DEPTH(16),
        .ADDR_WIDTH(4)
    ) DUT (
        .clk(clk),

        .rst_n(rst_n),

        .wr_en(wr_en),

        .rd_en(rd_en),

        .data_in(data_in),

        .data_out(data_out),

        .full(full),

        .empty(empty),

        .count(count)
    );

    // =====

    // VERIFICATION INFRASTRUCTURE LAYER (Structural Tasks & Arrays)

    // =====
```

```

// Golden Reference Model Software Storage Matrix

reg [7:0] golden_queue [0:2047];

integer head_ptr;

integer tail_ptr;

integer idx;


// Operational and Scoreboard Accounting Counters

integer operations_count;

integer checks_passed;

integer checks_failed;


// Pseudo-Random Generation Distribution Seed Data Variable

integer verification_seed;


// Drive continuous 100MHz master validation clock trace loop (10ns step window)

always #5 clk = ~clk;


// =====
// CONCURRENT MONITOR LOOP (Samples on negedge to ensure absolute alignment)
// =====

always @(negedge clk) begin

    if (rst_n) begin

        // Predictor: If a write happens and FIFO isn't full, push to software queue

        if (wr_en && !full) begin

            golden_queue[tail_ptr] = data_in;

            tail_ptr = tail_ptr + 1;

        end


        // Evaluator: If a read happens and FIFO isn't empty, pop and verify instantly

        if (rd_en && !empty) begin

            if (head_ptr != tail_ptr) begin

                if (data_out === golden_queue[head_ptr]) begin

                    $display("[SCB_PASS] Match Verified! Expected: 0x%0h, Observed: 0x%0h", golden_queue[head_ptr], data_out);

                    checks_passed = checks_passed + 1;

                end else begin

                    $display("[SCB_FAIL] Mismatch Detected! Expected: 0x%0h, Observed: 0x%0h", golden_queue[head_ptr], data_out);

                    checks_failed = checks_failed + 1;

                end

            end

        end

    end

end

```

```

        end

        head_ptr = head_ptr + 1;

    end

end

operations_count = operations_count + 1;

end

end

initial begin

    // Initialize Variable Index Pointers

    head_ptr    = 0;

    tail_ptr    = 0;

    operations_count = 0;

    checks_passed  = 0;

    checks_failed  = 0;

    verification_seed = 56214;


    // PRE-INITIALIZE THE ENTIRE GOLDEN QUEUE TO ZERO (Turns your waveform fully green!)

    for (idx = 0; idx < 2048; idx = idx + 1) begin

        golden_queue[idx] = 8'h00;

    end


    // Initialize Physical Pin States to Baseline Starting Conditions

    clk    = 0;

    rst_n  = 0;

    wr_en  = 0;

    rd_en  = 0;

    data_in = 0;


    // Apply Power-On Reset Delay

    #15;

    rst_n = 1;

    $display("[SIM_INITIALIZED] Global system power-on hardware reset released cleanly.");


    // Execute Constrained-Random Bus Traffic Pipeline Sweeps

    run_constrained_random_test(1000); // Process 1,000 randomized operations


    // Compile and Print Automated Functional Summary Report

```

```

$display("\n=====");
$display("      SYNCHRONOUS FIFO VERIFICATION REPORT      ");
$display("=====");
$display("[TOTAL STRESS CYCLES EVALUATED] : %0d Operations", operations_count);
$display("[AUTOMATED SCOREBOARD PASSES] : %0d Data Matches", checks_passed);
$display("[CRITICAL REGISTER MISMATCHES] : %0d Fault Errors", checks_failed);
$display("=====\\n");

if (checks_failed == 0) begin
    $display(">> [TEST BENCH STATUS]: SUCCESS! 100%% Functional Data Integrity Achieved.");
end else begin
    $display(">> [TEST BENCH STATUS]: FAILED! Logic mismatch errors tracked in netlist.");
end

$finish;
end

// Task: Pseudo-Random Stimulus Driver Loop (Generator/Driver Emulation)
task run_constrained_random_test(input integer num_items);
    integer item;

    reg [7:0] rand_write_probability;
    reg [7:0] rand_read_probability;

    begin
        for (item = 0; item < num_items; item = item + 1) begin
            @(posedge clk);

            #1; // Drive values 1ns after posedge (Standard Industry Skew)

            // Generate randomized stimuli variables

            rand_write_probability = $dist_uniform(verification_seed, 0, 100);
            rand_read_probability = $dist_uniform(verification_seed, 0, 100);

            // Apply a 60% workload probability distribution profile
            wr_en = (rand_write_probability < 60) ? 1'b1 : 1'b0;
            rd_en = (rand_read_probability < 60) ? 1'b1 : 1'b0;
            data_in = $dist_uniform(verification_seed, 0, 255);

            end

            @(posedge clk); // Allow final trailing operation window

        end

    endtask
endmodule

```